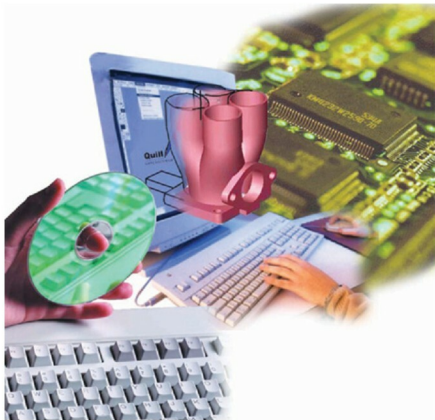


IN-TARGET TEST AUTOMATION AND TEST QUALITY TOOLS

New-generation embedded test tools enable software developers to be confident of delivering high-quality software, making a significant contribution to new products that work first time



■ **MAGNUS UNEMYR AND MARK MORAN**

Releasing a product with bugs is potentially very expensive, especially considering the cost of field upgrades, recalls and repairs. Less quantifiable, but equally important is the damage done to the reputation and consequent loss of customer-goodwill by any company. Notwithstanding, many embedded products are indeed released without all the necessary

and desirable testing. Consequently, software quality is receiving increasing focus in embedded application development.

Most existing software testing tools can only execute the tests on a PC, the use of which is limited when testing an embedded application. The actual system has numerous differences including hardware interfaces, timing issues and memory constraints among others. Therefore it is important to run as many software tests as possible

on the actual embedded hardware to minimise inter-platform differences.

The challenge is to find automated in-target software testing tools that can be easily integrated into developers' embedded development environments. These tools should be able to analyse the application source code, auto-generate a test suite and automatically run it on the target board. However, a test report showing 100 per cent success may not be sufficient as the test procedure itself may not have been applied to more than a small fraction of the code. Understanding the quality of test procedures thus becomes critical in judging whether a product is ready for release.

However, new and advanced tools for embedded test automation and test quality measurement are now emerging.

Unit testing

A key element of the software testing process is 'unit testing.' Unit tests call up 'C' programming functions with different combinations of input parameter values to drive the code through different execution paths. Other than for trivial functions, writing unit tests by hand is labour-intensive, costly, tedious, and might leave many possible execution paths untested. A detailed analysis of how input parameters drive the code is therefore required, so that important execution paths throughout a function are actually exercised by a test suite.

Another challenge faced by development teams is maintaining the synchronisation of unit tests with code under development. This problem